

Listeners de Widgets Android — Kotlin

Guía comparativa: cuándo usar Lambda (SAM) vs Object Expression · Basada en código real con ViewBinding

Widget	Listener / Método	Tipo (Lambda / Object)	¿Por qué?	Ejemplo en Kotlin	Métodos que implementa
Button	setOnClickListener	Lambda ✓	Solo tiene un método (onClick). SAM → lambda.	binding.aceptar.setOnClickListener { view -> // acción }	onClick(view: View)
Button	setOnLongClickListener	Lambda ✓	Un solo método (onLongClick). Devuelve Boolean.	binding.btn.setOnLongClickListener { true // consumido }	onLongClick(view: View): Boolean
CheckBox	setOnCheckedChangeListener	Lambda ✓	Interfaz CompoundButton.OnCheckedChangeListener tiene un solo método → lambda SAM.	binding.musica.setOnCheckedChangeListener { btn, isChecked -> // btn = CompoundButton, isChecked = Boolean }	onCheckedChanged(buttonView, isChecked: Boolean)
RadioGroup	setOnCheckedChangeListener	Lambda ✓	RadioGroup.OnCheckedChangeListener: un método → lambda.	binding.botonGroup.setOnCheckedChangeListener { group, checkedId -> val rb = findViewById(checkedId) }	onCheckedChanged(group: RadioGroup, checkedId: Int)
RadioButton (individual)	setOnCheckedChangeListener	Lambda ✓	Hereda de CompoundButton. Un método → lambda.	binding.rb.setOnCheckedChangeListener { _, isChecked -> }	onCheckedChanged(buttonView, isChecked: Boolean)
SeekBar	setOnSeekBarChangeListener	Object expression	Tiene 3 métodos obligatorios. No es SAM → se necesita object expression.	binding.seekBar.setOnSeekBarChangeListener(object : SeekBar.OnSeekBarChangeListener { override fun onProgressChanged(sb: SeekBar?, progress: Int, fromUser: Boolean) {} override fun onStartTrackingTouch(sb: SeekBar?) {} override fun onStopTrackingTouch(sb: SeekBar?) {} })	onProgressChanged(seekBar, progress, fromUser) onStartTrackingTouch(seekBar) onStopTrackingTouch(seekBar)
Switch / ToggleButton	setOnCheckedChangeListener	Lambda ✓	Hereda CompoundButton. Un método → lambda.	binding.miSwitch.setOnCheckedChangeListener { _, isChecked -> if (isChecked) { /* ON */ } else { /* OFF */ } }	onCheckedChanged(buttonView, isChecked: Boolean)
Spinner	onItemSelectedListener	Object expression	AdapterView.OnItemSelectedListener tiene 2 métodos → object expression.	binding.países.onItemSelectedListener = object : AdapterView.OnItemSelectedListener { override fun onItemSelected(parent: AdapterView<*>?, view: View?, pos: Int, id: Long) {} override fun onNothingSelected(parent: AdapterView<*>) {} }	onItemSelected(parent, view, position, id) onNothingSelected(parent)
ListView	setOnItemClickListener	Lambda ✓	AdapterView.OnItemClickListener: un método → lambda SAM.	binding.miLista.setOnItemClickListener { parent, view, position, id -> val item = parent.getItemAtPosition(position) }	onItemClick(parent, view, position: Int, id: Long)
ListView	setOnItemLongClickListener	Lambda ✓	Un método, devuelve Boolean → lambda.	binding.miLista.setOnItemLongClickListener { _, _, pos, _ -> true }	onItemLongClick(parent, view, pos, id): Boolean
EditText	addTextChangedListener	Lambda / Object	TextWatcher tiene 3 métodos. Con Kotlin Extensions (doOnTextChanged) es lambda; sin ellas, object expression.	// Lambda (KTX) binding.nombre.doOnTextChanged { text, _, _, _ -> } // Object expression (sin KTX) binding.nombre.addTextChangedListener(object : TextWatcher { override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int, after: Int) {} override fun onTextChanged(s: CharSequence?, start: Int, before: Int, count: Int) {} override fun afterTextChanged(s: Editable?) {} })	beforeTextChanged(s, start, count, after) onTextChanged(s, start, before, count) afterTextChanged(s: Editable?)
ImageView / cualquier View	setOnClickListener	Lambda ✓	View.OnClickListener: un método → lambda.	binding.miImagen.setOnClickListener { /* clic */ }	onClick(view: View)

Widget	Listener / Método	Tipo (Lambda / Object)	¿Por qué?	Ejemplo en Kotlin	Métodos que implementa
ViewPager2	registerOnPageChangeCallback	Object expression	ViewPager2.OnPageChangeCallback es clase abstracta con 3 métodos opcionales. Siempre object expression (o subclase).	binding.viewPager.registerOnPageChangeCallback(object : ViewPager2.OnPageChangeCallback() { override fun onPageSelected(position: Int) {} override fun onPageScrolled(pos: Int, offset: Float, offsetPx: Int) {} override fun onPageScrollStateChanged(state: Int) {} })	onPageSelected(position) onPageScrolled(pos, offset, offsetPx) onPageScrollStateChanged(state)
RecyclerView	addOnScrollListener	Object expression	RecyclerView.OnScrollListener: 2 métodos opcionales → object expression.	binding.rv.addOnScrollListener(object : RecyclerView.OnScrollListener() { override fun onScrolled(rv: RecyclerView, dx: Int, dy: Int) {} override fun onScrollStateChanged(rv: RecyclerView, newState: Int) {} })	onScrolled(recyclerView, dx, dy) on ScrollStateChanged(recyclerView, newState)
RatingBar	setOnRatingBarChangeListener	Lambda ✓	Un método → lambda SAM.	binding.ratingBar.setOnRatingBarChangeListener { rb, rating, fromUser -> binding.tvRating.text = rating.toString() }	onRatingChanged(ratingBar, rating: Float, fromUser: Boolean)
TimePicker	setOnTimeChangeListener	Lambda ✓	Un método → lambda SAM.	binding.timePicker.setOnTimeChangeListener { view, hour, minute -> }	onTimeChanged(view, hourOfDay, minute)
DatePicker	setOnDateChangeListener	Lambda ✓	Un método → lambda SAM.	binding.datePicker.setOnDateChangeListener { view, year, month, day -> }	onDateChanged(view, year, monthOfYear, dayOfMonth)
NumberPicker	setOnValueChangeListener	Lambda ✓	Un método → lambda SAM.	binding.numPicker.setOnValueChangeListener { picker, oldVal, newVal -> }	onValueChange(picker, oldVal, newVal: Int)
AutoComplete TextView	setOnItemClickListener	Lambda ✓	AdapterView.OnItemClickListener: un método → lambda.	binding.autoTV.setOnItemClickListener { parent, view, pos, id -> val selected = parent.getItemAtPosition(pos).toString() }	onItemClick(parent, view, pos, id)

Regla de oro: Si la interfaz tiene **un solo método abstracto (SAM)**, Kotlin permite usar una **lambda { }**. Si tiene **2 o más métodos** (aunque sean opcionales), necesitas un **object expression** o una subclase anónima. Las extensiones de KTX (doOnTextChanged, doOnClick...) encapsulan el object por ti, dando apariencia de lambda.